

Green Coding

Energy consumption in different programming languages and algorithms

Taha Khudher

Bachelor thesis

Main field of study: Computer engineering

Credits: 15hp

Semester/year: Spring/2024

Supervisor: Karl Pettersson

Examiner: Patrik Österberg

Course code/Registration Code: DT099G/DT-V24-G3-009

At Mid Sweden University, it is possible to publish the thesis in full text in DiVA (see appendix for publishing conditions). The publication is open access, which means that the work will be freely available to read and download online. This increases the dissemination and visibility of the degree project.

Open access is becoming the norm for disseminating scientific information online. Mid Sweden University recommends both researchers and students to publish their work open access.

I/we allow publishing in full text (free available online, open access):

☒

Yes, I/we agree to the terms of publication.

☐

No, I/we do not accept that my independent work is published in the public interface in DiVA (only archiving in DiVA).

Location and date: **Sundsvall, Sweden. (22/08/2024)**

Programme/Course: **Master of Science in Engineering: Computer Engineering / Bachelor Thesis.**

Name/Year of birth: **Taha Khudher/ 2002**

Abstract

This project investigates the impact of *Green Coding* on software development with the aim of reducing its environmental impact. The main objectives were to analyze algorithmic complexity, execution time and energy consumption in different programming languages. The results indicate that the energy consumption of an algorithm increases in conjunction with its algorithmic complexity. Execution time and energy usage are directly related, meaning performance optimization is in part an optimization of energy consumption. Different programming languages have different energy consumption values and each language performs differently. Go and Java showed similar performance with Java being slightly more efficient while C# was significantly more energy-consuming to the other two languages. Therefore, programming languages should be chosen carefully.

The scientific contribution of this work lies in its empirical analysis of energy consumption across various algorithms and languages, providing valuable resource for developers who wish to adopt greener coding practices. By demonstrating the environmental benefits of energy-efficient programming languages and algorithms, this work hopefully encourages the software industry to prioritize sustainability.

Keywords: Energy Consumption, Algorithms, Programming languages, C#, Java, Go.

Sammanfattning

Detta arbete undersöker effekten av *green coding* på mjukvaruutveckling med syfte att minska dess miljöpåverkan. Huvudmålen var att analysera algoritmisk komplexitet, exekveringstid och energiförbrukning i olika programmeringsspråk. Resultaten indikerar att energiförbrukning av en algoritm växer i samma takt som den algoritmiska komplexiteten. Exekveringstid och energianvändning är direkt relaterade vilket betyder att prestandaoptimering är i sin del en optimering av energiförbrukning. Olika programmeringsspråk har olika värden på energiförbrukning och varje språk presterar olika med Go och Java liknande prestanda men Java var lite effektivare och C# var alldeles för energikonsumerande jämfört med de två språken. Därmed bör programmeringsspråk väljas med omsorg.

Det vetenskapliga bidraget från detta arbete ligger i dess empiriska analys av energiförbrukning över olika algoritmer och språk, vilket ger en värdefull resurs för utvecklare som vill anta grönare kodningsmetoder. Genom att visa de miljömässiga fördelarna med energieffektiv programmeringsspråk och algoritmer önskar detta arbete mjukvaruindustrin att prioritera hållbarhet.

Nyckelord: Energikonsumtion, algoritmer, programmeringsspråk, C#, Java, Go

Table of Contents

Abstract	3
Sammanfattning	4
Table of Contents	5
1. Introduction	1
1.1 Background and motivation	1
1.2 Overall aim and problem statement	1
1.3 Scientific questions	2
1.4 Scope	2
1.5 Outline	3
2. Theory	4
2.1 Green Coding	4
2.2 Green Computing	4
2.3 Related work	5
2.3.1 Two level model for sustainable software development	5
2.3.2 Energy efficient coding and optimization techniques	6
3. Methodology	7
3.1 Scientific method description	7
3.2 Project/Work method description	8
3.2.1 Potential Issues	9
3.3 Project evaluation method	9
4. Choice of approach	11
4.1 Requirements	11
4.1.1 Requirement: Accurate Measurement of Energy Consumption	11
4.1.2 Requirement: Varied Algorithmic Complexity	11
4.1.3 Requirement: Selection of Programming languages	11
4.2 Comparison of Approaches	12
4.3 Chosen Approach	12
5. Implementation	14
5.1 Measurement Tools	14
5.2 Algorithms	14
5.2.1 QuickSort	15
5.2.2 Merge Sort	16
5.2.3 Selection Sort	17
5.3 Programming languages	19
5.4 Measurement/Evaluation setup	19
6. Results	20
6.1 Measurement results	20
6.1.1 Java measurements result	20
6.1.2 Go measurements result	22

6.1.3 C# measurements result	24
6.1.4 Language Comparison	26
7. Discussion	29
7.1 Analysis and discussion of the results	29
7.2 Project/Work method discussion	30
7.3 Scientific Discussion	31
7.4 Consequence analysis/Recommendation	32
7.5 Ethical and societal discussion	32
8. Conclusions	33
8.1 Future work	34
References	35

1. Introduction

The increased threat from carbon dioxide emissions and the greenhouse effect has heightened the need for innovative solutions to reduce environmental impact and promote sustainable development. Green programming needs to be used in order to achieve a smaller impact on the environment. What has been established over time is that there have been several climate crises around the world, which results in some continents being affected more than others [1]. What is mentioned is that it affects the economy, health and human societies. This is because humans are responsible for all emissions that affect these aspects above [2]. The focus of this project is to gain greater insights into how software has a negative environmental impact and how different aspects can help reduce this impact through sustainable software development. This project highlights the field of green programming as a potential key component for addressing these challenges.

This proposal is informed by my insights and the objectives of Knowit, the company I am making the project along to aid the company in their efforts to explore green coding.

1.1 Background and motivation

The greenhouse effect is a topic garnering increasing attention each year due to the exponential rise in emissions over the past century. Since 1950, carbon dioxide emissions have increased from 6 trillion tons to over 37 trillion tons in 2021, representing a surge of over 620 percent. These challenges will continue to escalate unless countermeasures are implemented. There are no significant, clear-cut solutions; rather, it requires small changes on various fronts to make the world a greener place. Software constitutes a significant part of human daily life, with many tasks and activities digitized. This underscores the importance of initiating change in the software development process. By employing various strategies, developers can reduce the hardware resources required for a system or application to function. This prompts the question: what strategies should be adopted, and how should they be practically implemented?[3].

1.2 Overall aim and problem statement

This project aims to explore further information on the subject of green coding and to identify one or more ways to create a more sustainable and environmentally friendly solution for software development while maintaining

good performance levels. By cutting down the problem to smaller problems and finding relevant and practical solutions that benefit developers in writing code that consumes less resources contributing to a positive environmental impact. In a world filled with concern about rapidly changing climate, where many species face extinction, oceans are acidifying, and many parts of the world are becoming uninhabitable, every small change is significant. According to the Paris Agreement, humans must strive to reduce carbon dioxide emissions by 45 percent by 2030. Everyone on Earth must collaborate to achieve this goal, including software developers, and changes must occur immediately.

1.3 Scientific questions

1. How does algorithmic complexity affect energy consumption of an algorithm?
2. What is the correlation between execution time and energy usage?
3. Do different programming languages consume more energy for the same algorithm?

1.4 Scope

This project will be limited to the software development process and how energy consumption can be reduced by writing more efficient code and using more effective tools. The framework tools that will be used will consist of programming languages and algorithms. The work will revolve around algorithms in specific and will explore different types of algorithms with different time complexity and data amounts to get a deeper understanding of the correlation between time complexity, execution time and power usage. These distinctions have been set up in order to get more in-depth work with a focus on development. With the short time frame assigned for the project, it is important not to make the work too extensive. By examining the behavior of power consumption under different conditions the project can give a deeper insight into how small changes in the software development process can achieve great environmental benefits. The timeframe of the work was limited leading to a smaller set of programming languages being chosen to save both writing time and experimental time. The chosen programming languages are all used in the same field of computer science, making it less time consuming to

compare the differences by removing many factors such as different abstraction levels and architectures.

1.5 Outline

Chapter 2 describes the theory and how to think in a greener manner. The theory section will name two important parts: the first one is green coding and the second part is green computing. Related work will also be mentioned in this section. Chapter 3 describes the chosen methodology. The quantitative method will be applied in this section, but also as a result of an experimental research design. There will be two variables: a dependent variable and an independent variable. Chapter 4 describes choice of approach where the selected programming languages and algorithms will be described. It will also mention how time and energy will be measured and the tools needed to get accurate measurements. Chapter 5 describes the implementation and all the technical details of the project. Here will all the components be highlighted such as the programming languages and algorithms. Chapter 6 will describe the results that are presented in various tables and figures. It will show all the tests on the selected programming languages Java, Go and C#. It will also contain different tools that have been used to get this result. Chapter 7 will describe in detail what the results show and answer the project's question.

2. Theory

This chapter is going to cover all the theory and background information needed to understand the problem and the problem motivation. The first part will give a short introduction of Green Coding and sustainable software development and explain what it is and what benefits it brings to work and think in a green manner while developing effective and efficient code. The second part is about Green Computing and will give a greater understanding on how computing devices in our day to day life have a negative environmental impact. This chapter will also look into some related work and discuss how this project is similar or different to existing efforts to reduce power consumption within the software development process.

2.1 Green Coding

Green coding, Sustainable Software Engineering or Green Software are all descriptions of one larger problem. Making software that has as little environmental impact as possible. Software is a part of computing that has not been criticized enough from an environmental standpoint. Developers have always valued human-time as the most valuable resource, leading a lot of developers to develop products with the help of libraries or frameworks to save time and resources. The problem with such an approach is that we download tools that are much larger than needed and write inefficient code to deploy as fast as possible which leads to a product that is too big and inefficient for its purpose. Instead of writing more energy efficient algorithms and better software humans buy newer hardware capable of handling the new software which leaves old hardware practically useless. [4]

2.2 Green Computing

Research in Green Computing has been evolving and more studies introducing concepts to reduce greenhouse gas emissions have been introduced due to the growing threat of global warming. In a world where digital technologies are a major part of human life the importance of such studies has increased. Each personal computer produces about a ton of CO₂ per year in electricity consumption assuming that the energy is not clean energy.[5] This is without taking into account the emissions during the manufacturing of these computing devices. Green computing involves manufacturing, designing, using and disposing of computers, servers and associated subsystems. The ICT sector is responsible for about 7% of global electricity consumption (in the year 2022)

according to the European Commission and is expected to increase to 13\% by 2030[6][7].

2.3 Related work

This chapter will discuss related and relevant work or research earlier performed and how it can be used to aid the project and how it will be different.

2.3.1 Two level model for sustainable software development

Sara Mahmoud and Imiaz Ahmad's article *A green model for sustainable software engineer*. [8] focuses on developing a two-level green software model to enhance sustainability throughout the software life cycle. The first level provides guidelines for building green software through a mix of sequential, iterative and agile development methods. This level includes various stages such as requirements gathering, where developers evaluate the necessity and potential energy efficiency of new systems. Design an implementation focusing on efficient data structures, algorithms and system architecture. Testist which not only ensures functionality but also assess environmental impact and fault tolerance. The green analysis stage measures energy metrics like CPU usage, while the usage stage emphasizes energy-efficient resource management based on user interaction. Maintenance ensures ongoing sustainability, even with new feature additions, and the disposal stage advocates for environmentally responsible recycling of outdated hardware and software.

The second level emphasizes software tools that enhance energy efficiency, covering operating system frameworks, fine-grained computing techniques, performance metrics, energy allocation algorithms and virtualization methods. These tools help software designers optimize energy consumptions, thus fostering environmental sustainability.

This project builds on Mahmoud and Ahmad's work by focusing on algorithms and choice of programming language. By diving deeper into these areas the project aims to get a deeper understanding into how small change in every part of the development process can make a big difference in the end.

2.3.2 Energy efficient coding and optimization techniques

The article *Analysis of Energy consumption and optimization Techniques for writing Energy-Efficient code* [9] written J.Coral-Garcia et. discusses the impact of algorithm optimization on energy consumption and provides evidence that manual optimization by software developers can lead to more energy savings compared to relying solely on compiler optimizations. The summary talks about different topics with the key topics that will aid me in my work.

The research demonstrates that programmers can achieve greater energy savings by applying optimization techniques manually rather than depending on automatic optimization options provided by the GNU compiler collection(GCC) [9]. A set of techniques for writing energy-efficient code for Raspberry Pi devices is analyzed by the writers of the article showing the potential for energy saving when used. Twenty-five techniques were analyzed. Techniques such as cascaded function calls, exception handling and linear search are an example of the techniques mentioned for their significant energy savings when applied manually by the programmer. The result showed an energy save of up to 99.56% using these different techniques which gives my work an indicator that this project can have a great impact and that just some minimal optimizations can make a huge difference.

This article was a very good find and will aid the project in understanding what techniques have been tried and how they impact the quality and energy consumption of software. The project will be similar to the article in which it will use techniques and methods to further optimize and find more solutions to the problem of energy consumption in the field of software development. This article also has techniques that could be implemented in this project and further investigated to get a greater understanding of energy consumption in computers and software and what leads to the most energy wastage.

3. Methodology

This chapter will further describe the method chosen to answer the scientific question. The chapter will give an insight into how the work will be done step by step and how the process is expected to generate the results needed to answer the scientific questions.

3.1 Scientific method description

For this project a quantitative method will be used, the nature of quantitative methods aligns with the objective of measuring energy consumption of an algorithm under various conditions such as execution time, time complexity, amount of data and what programming language that is used [10].

The chosen methodology will follow an experimental research design inspired method where a dependent variable (energy consumption) will be used and manipulated with the help of the independent variables (execution time, algorithmic complexity, programming language, data volume).

Experimental research design involves controlled experiments where one or more independent variables are manipulated to observe the effects they have on the chosen dependent variable. This approach gives an understanding of the cause-and-effect relationship between the dependent and independent variables. This manipulation gives a more clear image on what factors affect the dependent variables and what does not which makes it easier to include or exclude these variables in future work [10].

The method chosen will consist of six phases. The phases are the following:

- 1. Defining the research problem and objectives.**
- 2. Designing the experiment.**
- 3. Implementing the experiment.**
- 4. Conducting the experiment.**
- 5. Analyzing the data.**
- 6. Report the findings.**

To answer the first and second scientific questions an experiment will be conducted. The dependent variable is energy consumption (E) and the independent variables are algorithms with different time complexities, execution time and amount of data to handle. Analysis will be conducted to monitor how the energy consumption grows when the time complexity and

data amount of an algorithm grows and how execution time correlates to the dependent variable.

Answering the third question includes implementing the chosen algorithms in different programming languages. The dependent variable is energy consumption and the independent variables are programming language, algorithm and data size.

3.2 Project/Work method description

The working method for this project will follow a design science inspired structure which is divided into several phases where each phase serves the purpose of systematically achieving the goals and answering the scientific question in order to make the results public in the latter phases.

Identify Problems: The first phase requires defining the problem statement and showcasing its importance within the domain. This phase highlights the significance of energy efficiency in software development, especially for large-scale and long-running applications where a single algorithm executes thousands of times.

During this phase a literature review is conducted to identify related work and identify gaps in the current available research. This phase gives the possibility to define the scope of the research problem and what this project will focus upon.

Define Objectives: The second phase involves establishing specific objectives and requirements for the research. This includes outlining the scientific questions, developing hypotheses and defining what experiments will be set and what the experiments will measure.

Design Development: The third phase focuses on the planning and building of the experimental setup. The algorithms to be tested will be chosen, the programming languages needed, the different sizes of data and the tools for measuring.

Demonstration: In the fourth phase is a critical phase where all the experiments will be executed to collect the data needed. This phase needs good documentation and accuracy to ensure that the experiments can be reproduced.

Evaluation: The fifth phase will analyze the collected data to evaluate the impact of the identified factors on energy consumption. Here the results of the experiments will be compared and conclusions will be drawn based on statistical analysis. This phase will also compare the result to the hypothesis if

there is any.

Communication: Finally the findings will be compiled into the project report. This is the phase where the work is presented to academic and professional stakeholders. In this phase a set of visual illustrations will be created to aid in showing the result in a visual and comprehensive manner.

Justification: The design science research model is well-suited for this project as it emphasizes both the experimental setup and the evaluation of its effectiveness.

3.2.1 Potential Issues

Measurement accuracy:

Measuring on a computer with the help of software can be challenging due to the computer having background processes and other noise that can interfere with the measurements even if only the given algorithm and its process are measured. To accommodate for this issue the same experiment will be ran five times each and the average will be calculated to get a more accurate result.

Comparability

Different programming languages may have an inherent difference in energy efficiency. Each programming language has a specific purpose which makes it hard to judge them on one parameter only which in this case is algorithms. To accommodate for this issue a set of languages within a specific domain will be chosen.

Data volume

Large datasets might introduce noise or require significant processing power which is hard to get on a portable device. These algorithms often run on large servers with much more robust hardware which can handle such tasks easier. The portable device used for these experiments has limited physical memory which makes it vulnerable to the switch to swap memory which can reduce performance and increase energy consumption. Proper data management and scaling experiments appropriately can address this problem.

3.3 Project evaluation method

To evaluate the overall success of the project a set of key evaluation criterias will be reviewed to get a deeper insight into how the project planning, execution and outcome. The achievement of the scientific goals and to answer the scientific question is a key criteria for the project to be regarded as

successful. The results should be comprehensive enough to be discussed. The effectiveness of the chosen methodology will also be discussed and analyzed to evaluate if it was the right choice for the project. Other important factors that will be evaluated are personal growth and impact of the findings.

4. Choice of approach

This chapter will lift and discuss the different choices and options that had to be made to approach the problem. This chapter will discuss each requirement, the different choices within them, differences between each approach, what pros and cons there is to each choice and which choice is the best fit for the project.

4.1 Requirements

4.1.1 Requirement: Accurate Measurement of Energy Consumption

Hardware-based Measurement: Using physical devices like power meters to directly measure the power consumption of the machine running the experiments.

Software-based Measurement: Utilizing software tools and utilities that provide energy consumptions metrics based on system data.

4.1.2 Requirement: Varied Algorithmic Complexity

Classic Algorithms with Known Complexities: Implementing and testing well known algorithms with different complexities(e.g. Sorting algorithms , search algorithms)

Real-world Scenarios: Using algorithms that are commonly used in real-world scenarios such as data processing, machine learning and searching.

4.1.3 Requirement: Selection of Programming languages

Broad Evaluation: Comparing different programming languages with different purposes such as languages for statistics such as R, low level languages for embedded devices such as C/C++ and languages that run on the web such as Javascript. This gives a deeper understanding into how different layers of abstractions and language architecture can lead to increased energy consumption in specific tasks.

Focused Evaluation: Comparing energy consumption between Languages with similar functionalities and purposes to get a deeper understanding of how different programming languages with the same end goal purpose can be more or less energy efficient in a set of specific tasks. In this approach languages

such as Java, C# and Golang are compared side by side executing the same operations.

4.2 Comparison of Approaches

To analyze the identified requirements and compare different approaches, this subchapter will evaluate the benefits and drawbacks of each method.

Accurate measurement of energy consumption can be approached using either hardware-based or software based measurement methods. Hardware-based measurement offers the benefit of direct measurement and high accuracy, but is expensive, requires physical setup and may not isolate specific processes. On the other hand software-based measurement is cost-effective and can measure specific processes with ease, although it might be less accurate due to system noise and relies on software accuracy which can be hard to calibrate correctly.

Varied algorithmic complexity can be assessed using either classic algorithms with well known complexities or through real-world scenarios. Classic algorithms are well-documented and easy to compare in a controlled environment but they may not reflect real-world usage and have a limited scope. Real- World scenarios provide practical application insights and diverse situations, but are harder to generalize and control due to variability.

Selection of programming languages involves making a choice between a board or focused evaluation. A broad evaluation offers comprehensive and diverse insights across different languages, but is more complex to set up and control variables such abstraction layers and the scope of the language. A focused evaluation allows for an in-depth analysis of the same types of languages creating a more accurate comparison where the languages have similar architecture and similar functionalities.

4.3 Chosen Approach

Based on the analysis of the identified requirements and approach alternatives the chosen approach involves using Software-based measurement tools due to their convenience and cost-effectiveness. Software-based measurements also give the possibility to measure just a single part or process of the system giving less room for noise and needing less calculation and measurements to get a result.

For Algorithms Classic algorithms with well-known complexities will be used for their ease of comparison and controller environments. Real-world scenarios can be hard to find due to companies and developers using different methods and needing different specifications for their models making it hard to find something that generalizes well.

For programming the approach chosen is a focused comparison choosing three languages close to each other, Java, C# and Go. All three chosen languages are general purpose and used for web development and backend services where such operations can be found. [15-17]

The statistics will be mentioned in more detail where the selected programming languages will be presented and how much these were used in the year 2023 by programmers. The first programming language mentioned is Java with 30.55% having used this. Then you can see that after that language comes C# which is also a programming language that is used extensively by a full 27.62%. Go is the program language that falls a little under but is very attractive among backend developers but where a full 13.24% use it [18].

This Approach aligns with the overall aim of the project, providing comprehensive insights into how energy consumption varies with algorithmic complexity, execution time and data volume across different programming languages. A total of 225 tests will be executed. By choosing the following approaches and the methodology the project will give valuable insights into the field of energy efficient software development.

5. Implementation

In this chapter all the technical details of the project will be explained. This begins with an overview of the project and all the components needed illustrated in figure 1.

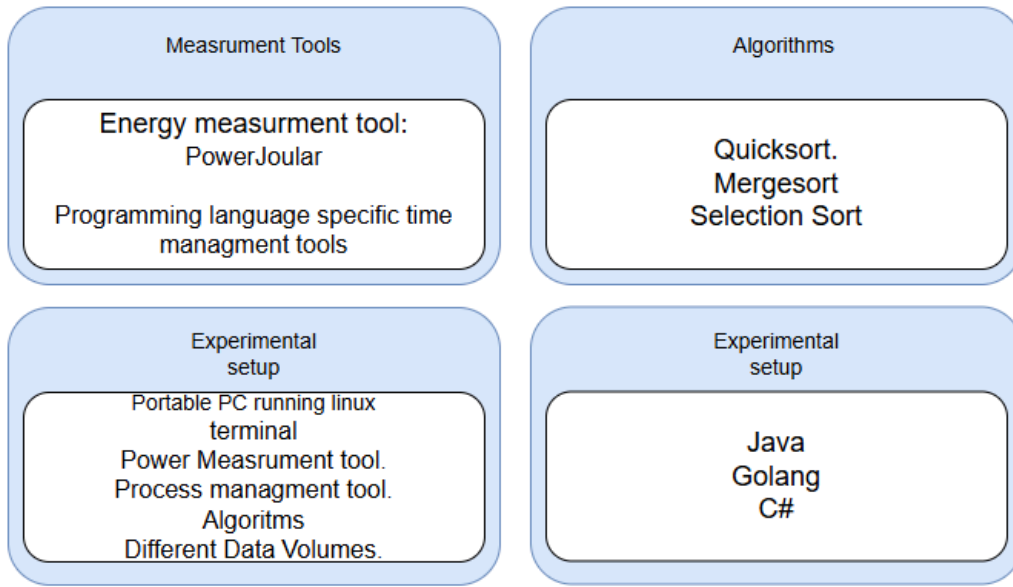


Figure 1: System overview

5.1 Measurement Tools

The first part of the overview in **figure 1** focuses on the measurement tools used for the project which includes an energy measuring tool and a time measurement tool. The energy measuring tool chosen is PowerJoular[11], an open source tool with the ability to specify a specific Process ID or Process name to measure a specific task run by the system. This tool makes it easier to get more accurate measurements and to exclude other tasks or processes that may interfere with the measurement. The time management measurement tools are built in or can be imported through libraries for each programming language. In C# and Java using the built in timer of the system with a function. In Golang the Time module is imported.

5.2 Algorithms

The second part of **figure 1** is the chosen sorting algorithms. Each algorithm chosen is chosen with the algorithmic complexity in mind with each algorithm having different complexities to measure and compare the difference made by

the algorithmic complexity on the energy consumption. The algorithms chosen are listed below.

5.2.1 QuickSort

QuickSort is an efficient sorting algorithm that follows a divide and conquer paradigm. The algorithm works by choosing a pivot element and partitioning the rest of the elements into two subarrays. One array holds the elements smaller than the pivot and the other containing larger elements. The pivot is then placed in the right place in the array. This process is recursively applied to the subarrays too leading to a time complexity of $O(n \log n)$.

Pseudocode of QuickSort[12]. Codes found in Appendix

```
function QUICKSORT(ARRAY, START, END)           (1)
  if START >= END then                           (2)
    return                                           (3)
  end if                                           (4)
  PIVOTINDEX = PARTITION(ARRAY, START, END)(5)
  QUICKSORT(ARRAY, START, PIVOTINDEX - 1)          (6)
  QUICKSORT(ARRAY, PIVOTINDEX + 1, END)           (7)
end function                                     (8)
```

```
function PARTITION(ARRAY, START, END)             (1)
  PIVOTVALUE = ARRAY[END]                         (2)
  PIVOTINDEX = START                               (3)
  loop INDEX from START to END                 (4)
    if ARRAY[INDEX] <= PIVOTVALUE                 (5)
      TEMP = ARRAY[INDEX]                         (6)
      ARRAY[INDEX] = ARRAY[PIVOTINDEX]            (7)
      ARRAY[PIVOTINDEX] = TEMP                     (8)
      PIVOTINDEX = PIVOTINDEX + 1                  (9)
    end if                                         (10)
  end loop                                       (11)
  return PIVOTINDEX - 1                          (12)
```

5.2.2 Merge Sort

Merge sort is a sorting algorithm following a divide and conquer paradigm. The algorithm splits the array into two halves recursively sorting each half then merges the halves together ensuring that all elements are sorted. Merge sort has a time complexity of $O(n \log n)$

Pseudocode of MergeSort[13]. Codes found in Appendix

```
function MERGESORT(ARRAY, START, END)           (1)
  if END - START + 1 == 1 then                   (2)
    return                                         (3)
  end if                                         (4)
  if END - START + 1 == 2 then                   (5)
    if ARRAY[START] > ARRAY[END] then           (6)
      TEMP = ARRAY[START]                       (7)
      ARRAY[START] = ARRAY[END]                 (8)
      ARRAY[END] = TEMP                         (9)
    end if                                       (10)
    return                                       (11)
  end if                                         (12)
  HALF = int((START + END) / 2)                  (13)
  MERGESORT(ARRAY, START, HALF)                  (14)
  MERGESORT(ARRAY, HALF + 1, END)                (15)
  MERGE(ARRAY, START, HALF, END)              (16)
end function
```

```

function MERGE(ARRAY, START, HALF, END)           (1)
  TEMPARRAY = new array[END - START + 1]           (2)
  INDEX1 = START                                     (3)
  INDEX2 = HALF + 1                                   (4)
  NEWINDEX = 0                                       (5)
  loop while INDEX1 <= HALF and INDEX2 <= END      (6)
    if ARRAY[INDEX1] < ARRAY[INDEX2] then           (7)
      TEMPARRAY[NEWINDEX] = ARRAY[INDEX1]           (8)
      INDEX1 = INDEX1 + 1                           (9)
    else                                             (10)
      TEMPARRAY[NEWINDEX] = ARRAY[INDEX2]           (11)
      INDEX2 = INDEX2 + 1                           (12)
    end if                                         (13)
    NEWINDEX = NEWINDEX + 1                         (14)
  end loop                                         (15)
  loop while INDEX1 <= HALF                         (16)
    TEMPARRAY[NEWINDEX] = ARRAY[INDEX1]             (17)
    INDEX1 = INDEX1 + 1                             (18)
    NEWINDEX = NEWINDEX + 1                         (19)
  end loop                                         (20)
  loop while INDEX2 <= END                         (21)
    TEMPARRAY[NEWINDEX] = ARRAY[INDEX2]             (22)
    INDEX2 = INDEX2 + 1                             (23)
    NEWINDEX = NEWINDEX + 1                         (24)
  end loop                                         (25)
  loop INDEX from 0 to size of TEMPARRAY - 1 (26)
    ARRAY[START + INDEX] = TEMPARRAY[INDEX] (27)
  end loop                                         (28)
end function

```

5.2.3 Selection Sort

Selection sort follows the greedy algorithm paradigm. The algorithm repeatedly finds the smallest element in the unsorted portion of the list and swaps it with the first element of the unsorted list. The process is repeated for the remaining of the unsorted portion until the list is sorted.

Pseudocode of SelectionSort[14]. Codes found in Appendix.

```
function SELECTIONSORT(ARRAY) (1)
  loop INDEX from 0 to size of ARRAY - 2 (2)
    MININDEX = 0 (3)

    loop INDEX2 from INDEX to size of ARRAY - 1 (4)
      if ARRAY[INDEX2] < ARRAY[MININDEX] then (5)
        MININDEX = INDEX (6)
      end if (7)
    end loop (8)
    TEMP = ARRAY[MININDEX] (9)
    ARRAY[MININDEX] = ARRAY[INDEX] (10)
    ARRAY[INDEX] = TEMP (11)
  end loop (12)
end function (13)
```

5.3 Programming languages

The third part of **Figure 1** examines the impact of different programming languages on energy consumption. The comparison is drawn between 3 languages. Java, C# and Golang, three famous and widely used languages used for many tasks. Through implementing all the algorithms mentioned in section 5.2 and measuring the energy consumption of the algorithms in these languages, conclusions can be drawn to the third scientific question.

5.4 Measurement/Evaluation setup

This section describes how the measurement and evaluation of the implementation were conducted. All the algorithms were implemented in the mentioned programming languages. The algorithms then were tested on each language at a time. Each algorithm will be executed with 5 different amounts of data volume, 500, 5,000, 50,000, 500,000 and 1,000,000 elements. Each combination of algorithm, data volume and programming language will be executed 5 times to calculate the average energy consumption over the 5 tests to get more accurate results and minimize the room for errors and noise. A total of 225 tests. When the program is executed the algorithm doesn't start immediately but waits for the user to input an enter for the test to start. This gives time to find the process and give its process ID with the help of HTOP resource management program for linux to the PowerJoular energy monitoring

tool to measure the energy consumption. When the user types enter and the algorithm starts a timer built into the programming language starts, counting the time it takes for the program to finish. With the time and energy consumption given the mean value is calculated and then documented.

6. Results

This chapter will analyze the selected material and answer the questions about how algorithmic complexity affects the energy consumption of an algorithm and if a connection can be seen between execution time and energy consumption. The material that will be put into practice is different programming languages and algorithms.

6.1 Measurement results

In the tables below the result of the measurements is presented. On the x-axis there is information on the data volume. On the y-axis each algorithm is presented. In each cell there is information about average execution time and Energy consumed in joules. The chapter also contains figures with graphs. The figures will be further explained later.

6.1.1 Java measurements result

Table 1. Java measured times and energy consumption in Joules

	<i>Data volume</i>				
	500	5000	50 000	500 000	1 000 000
Quick Sort	0.36 ms 0.57 mJ	5.88 ms 0.013 J	573.81 ms 7.43 J	43. 22 s 772 J	229 s 4122 J
Selection Sort	0.29 ms 0.444 mJ	5.8 ms 0.011 J	430 ms 6.8 J	42. 24 s 701 J	166 s 2988 J
Merge Sort	0.09 ms 0.091 mJ	0.37 ms 0.86 mJ	3.02 ms 8.37 mJ	20. 29 ms 90 mJ	34.68 ms 205 mJ

The graph shows a graphical view of the measured value for the algorithms executed in Java. See figure 2 and 3.

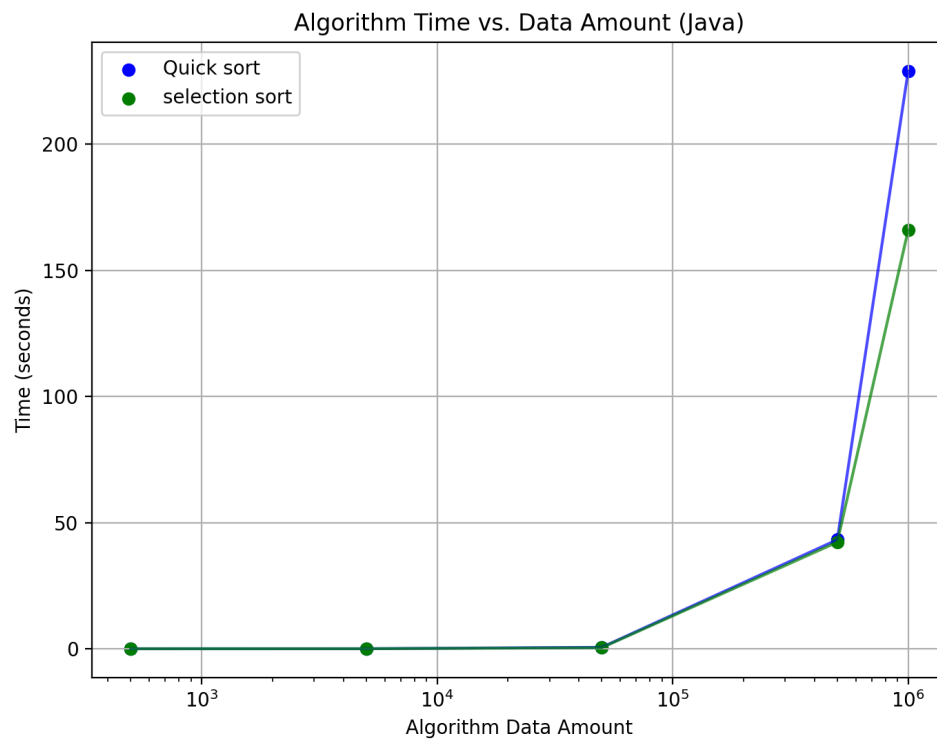


Figure 2: Plot of the Measured values for quicksort and selection sort in java.

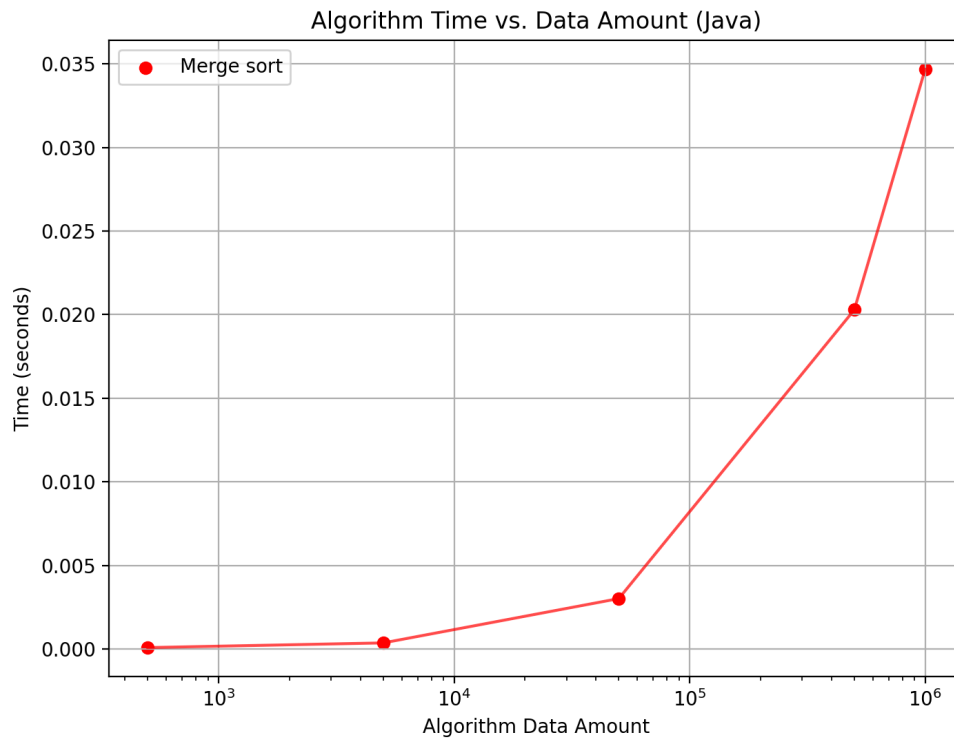


Figure 3: Plot of the Measured values for merge sort in java.

The graph below is a plot of the time on the Y-axis and the Energy in Joules in the x-axis. See figure 4.

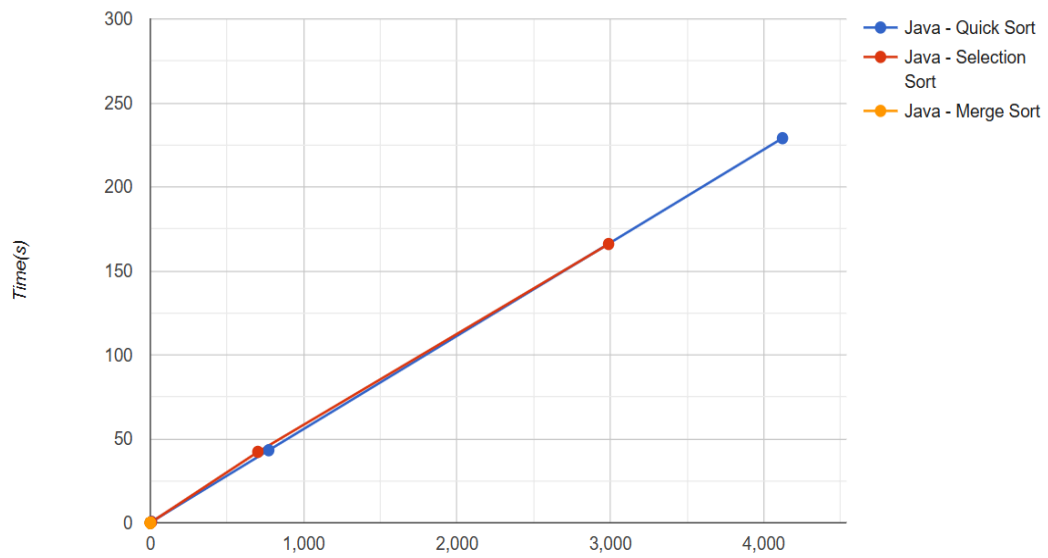


Figure 4: Plot of the Energy consumed and Time in Java.

6.1.2 Go measurements result

Table 2. Go measured times and energy consumption in Joules
Data volume

	500	5000	50 000	500 000	1 000 000
Quick sort	0.16 ms NaN	6. 82 ms 11.18 mJ	645 ms 10.7715 J	61 s 1037 J	247 s 4100 J
Selection sort	0.15 ms NaN	6. 94 ms 8.40 mJ	611 ms 9.40 J	61 s 983 J	245 s 3994 J
Merge sort	0.06 ms NaN	0. 46 ms NaN	4. 88 ms 7.76 mJ	34.12 ms 189 mJ	59 ms 425 mJ

The graph shows a graphical view of the measured value for the algorithms executed in Java. See figure 5 and 6.

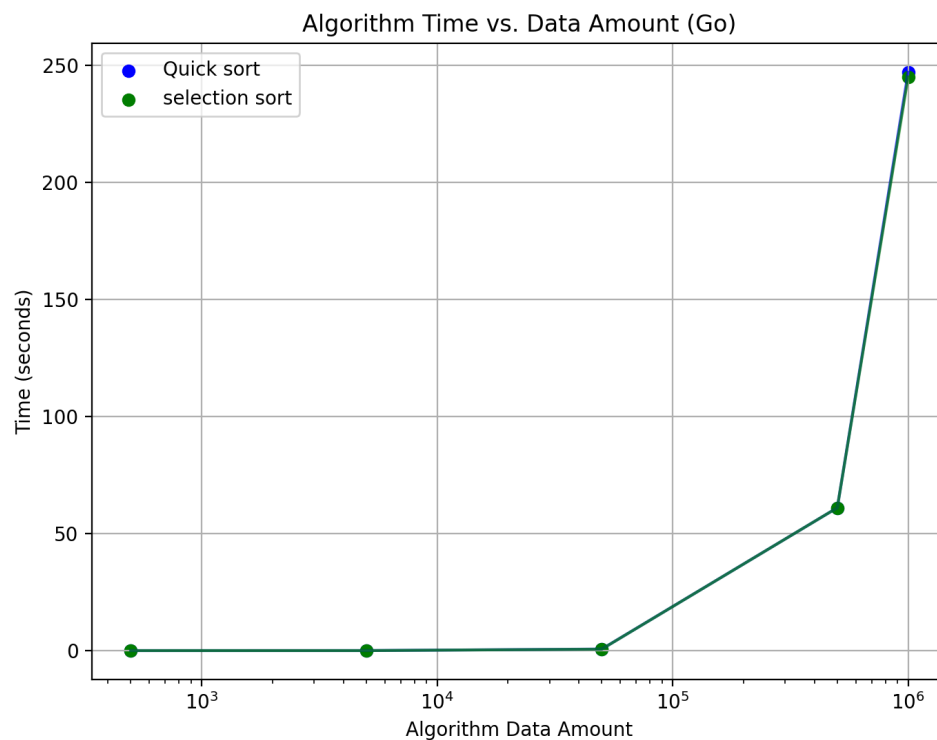


Figure 5: Plot of the Measured values for quicksort and selection sort in Go.

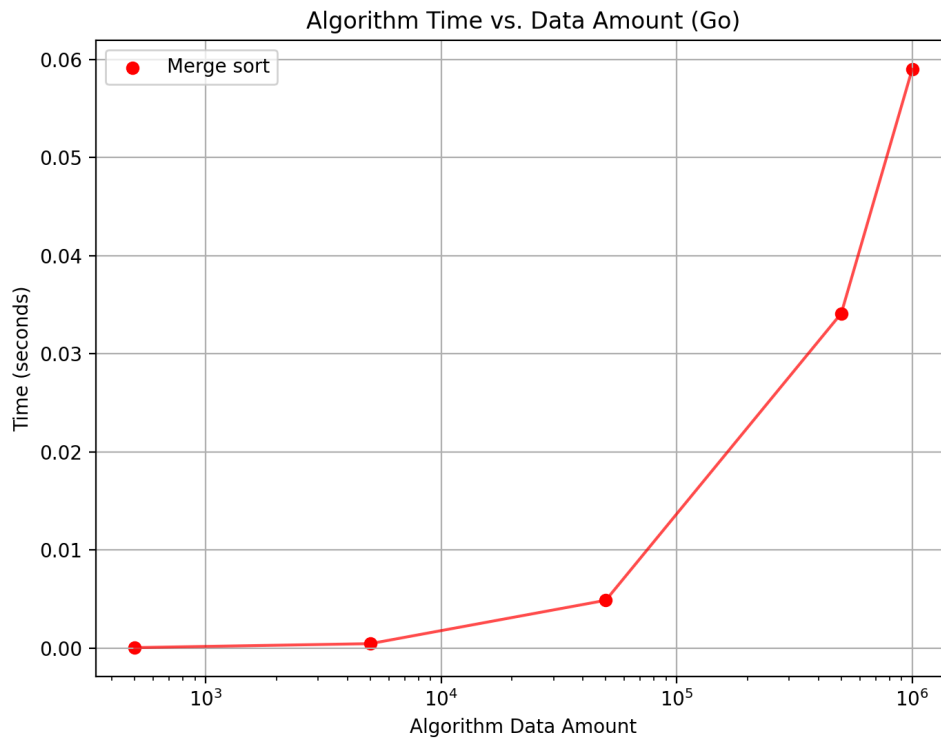


Figure 6: Plot of the Measured values for merge sort in Go

The graph below is a plot of the time on the Y-axis and the Energy in Joules in the x-axis. See figure 7.

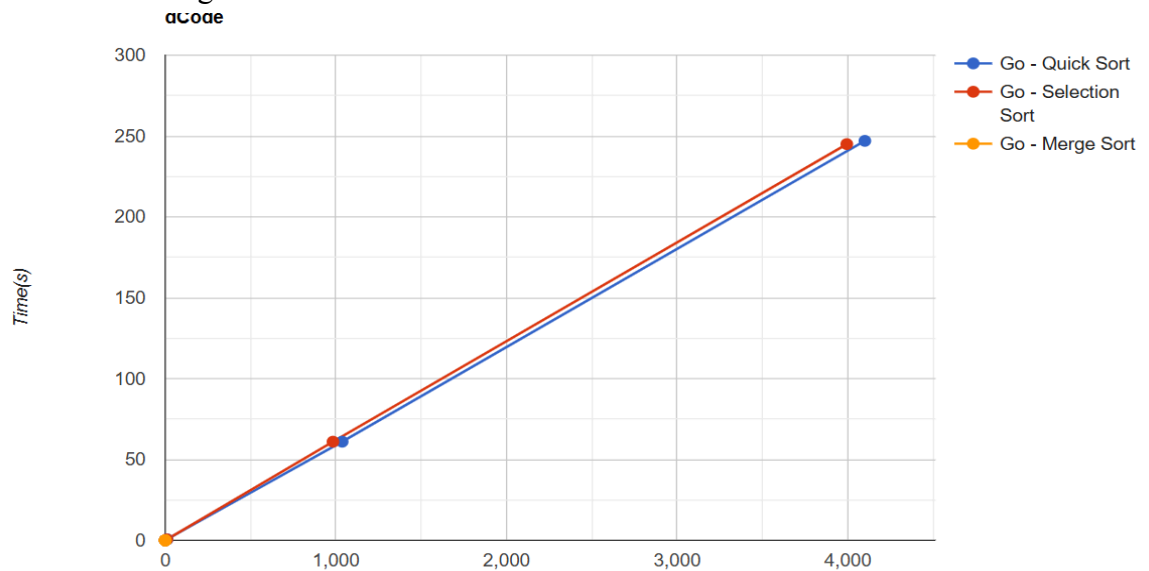


Figure 7: Plot of the Energy consumed and Time in Go.

6.1.3 C# measurements result

Table 3. C# measured times and energy consumption in Joules

	<i>Data volume</i>				
	500	5000	50 000	500 000	1 000 000
Quick sort	2.16 ms 2.57 mJ	27.32 ms 135.5 mJ	2.57 s 44.5 J	262 s 4533 J	1061 s 19100 J
Selection sort	1.97 ms 2.31 mJ	16.91 ms 37.7 mJ	1.37 s 23 J	136 s 2421 J	550 s 9790 J
Merge sort	1.9 ms 2.26 mJ	2.33 ms 4.24 mJ	6 ms 13.4 mJ	43 ms 187 mJ	80 ms 448 mJ

The graph shows a graphical view of the measured value for the algorithms executed in C#. See figure 8 and 9.

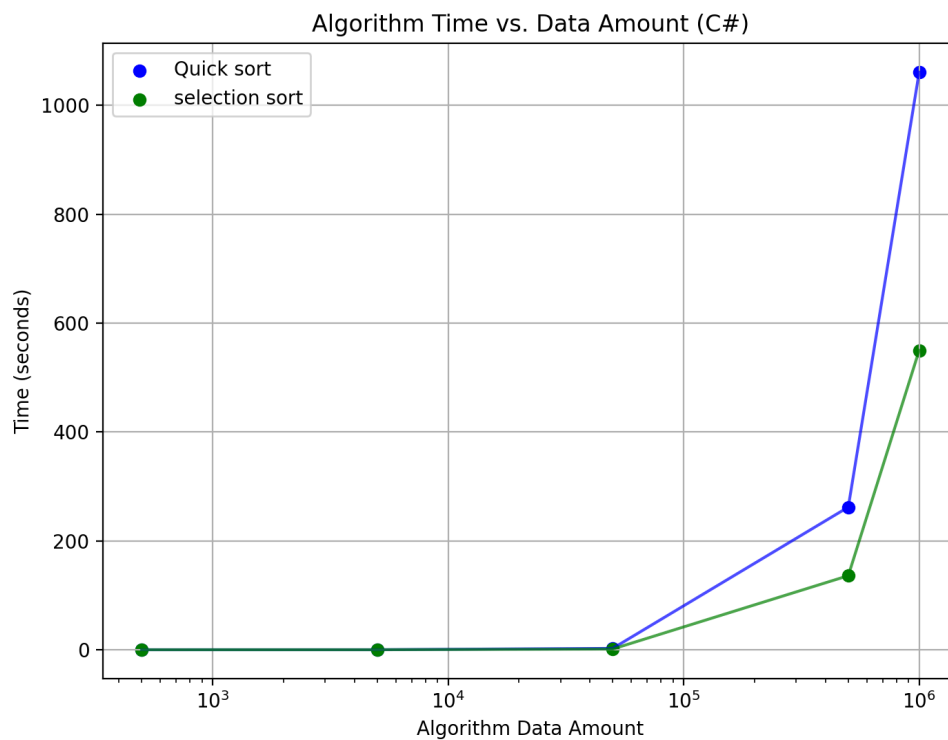


Figure 8: Plot of the Measured values for quicksort and selection sort in C#.

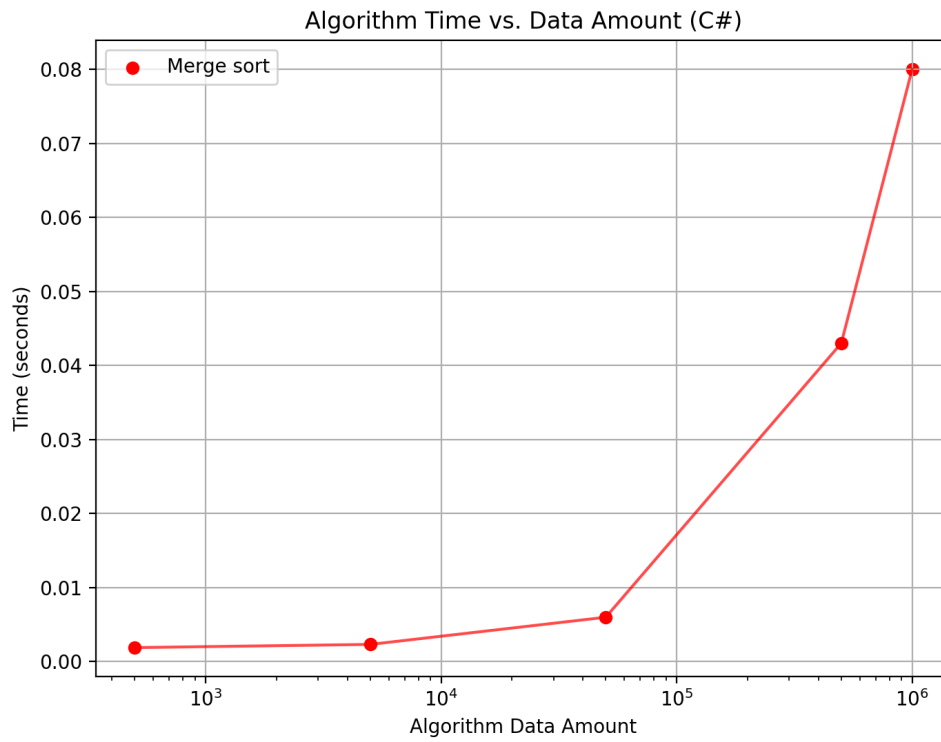


Figure 9: Plot of the Measured values for merge sort in C#.

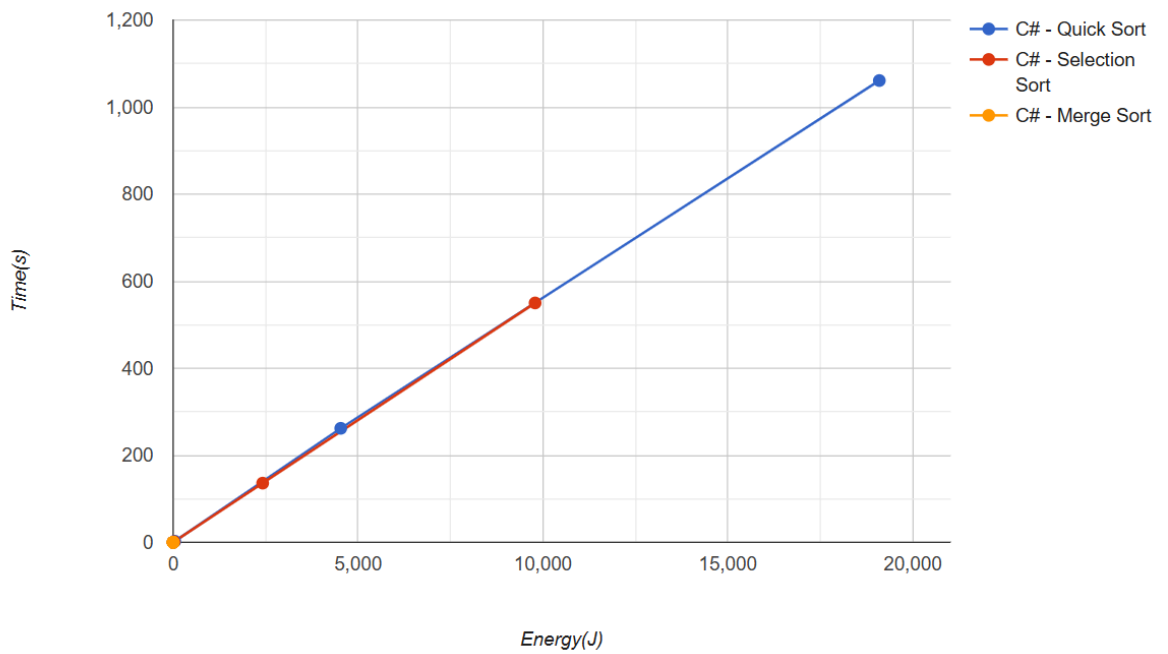


Figure 10: Plot of the Energy consumed and Time in C#.

6.1.4 Language Comparison

The following two graphs Show the average energy consumption per algorithm in each language. See figure 11 and 12.

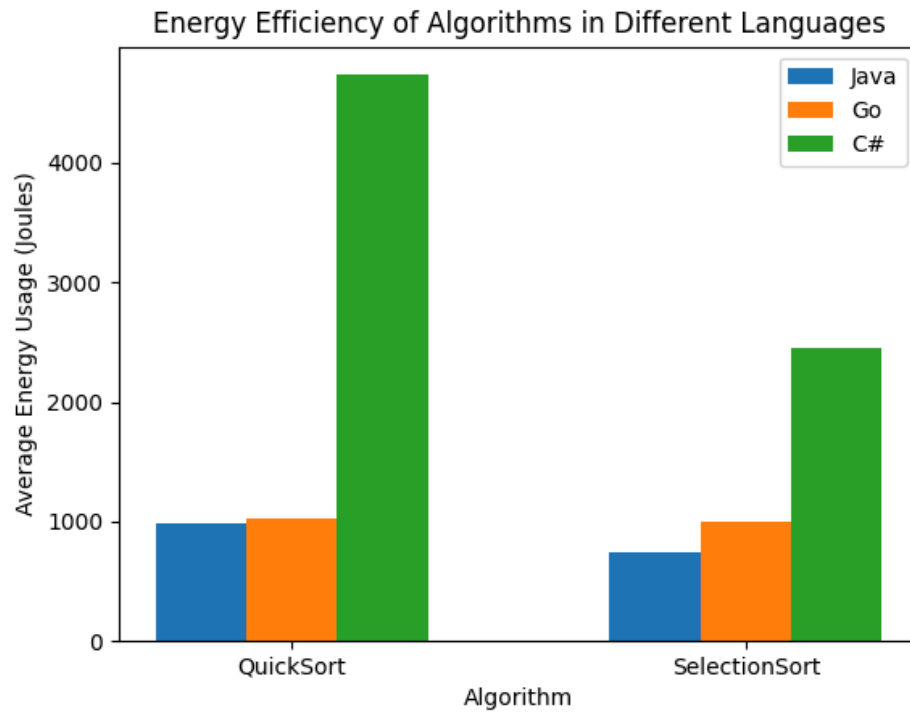


Figure 11: Chart of the average energy usage for Quicksort and Selection Sort in each language.

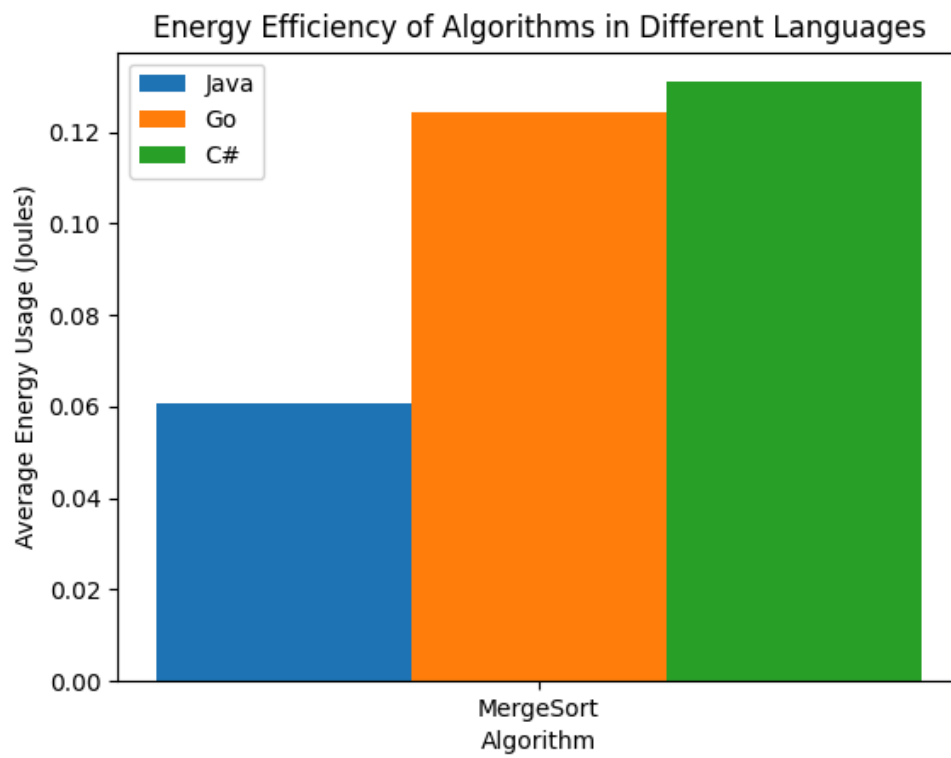


Figure 12: Chart of the average energy usage for Merge Sort in each language.

7. Discussion

In this section the findings of the project will be discussed and analyzed. The goal is to understand how different algorithms and programming languages impact energy usage. The measurements and experiments are going to be used to answer the scientific questions. This chapter will also reflect on the methodology employed and consider the broader scientific, ethical and societal impact of the project.

7.1 Analysis and discussion of the results

The results indicate significant variations in energy consumptions across different algorithms and programming languages. The results clearly shows a big difference between the energy consumption of C# compared to GO and Java. Java and Go had very similar performance where Java was just a bit more energy efficient compared to Go but C# on the other hand consumed a lot more energy and in some cases consumed 4 times more energy than the other two languages for the same tests. This suggests that the efficiency of language compilers and runtimes leave a great impact on energy usage.

There was a notable correlation between execution time and energy consumption. As seen in figures 3, 5 and 7 the correlation between time and energy usage is a linear correlation which is a 1 to 1 correlation. This correlation was expected due to the energy formula being $E(J) = P(W) * t(s)$. But the tests were implemented to see if factors such as CPU load could affect this correlation.

The algorithmic complexity showed a great difference in energy consumption as expected. The algorithmic complexity is a measurement into how the computational need grows when the amount of data increases. In the algorithms with a worst case algorithmic complexity $O(n^2)$ the graph clearly shows an exponential increase in energy consumption and execution time. The merge sort algorithm was an outlier on all 3 languages and finished very quickly with minimal energy consumption. The matter has been analyzed, no clear reason has been found as to how the performance was so much better, different tests and rewrites were conducted to insure that the results were correct but no indication of anything wrong was found. The difference between the algorithms could lie down to factors such as Compiler optimization, system resources and dataset characteristics but no reason for

such a difference in performance should exist. Although the energy consumption and execution time was low during the merge sort test the same patterns and correlations can be seen through the results.

These findings indicate that selecting an appropriate programming language and optimizing algorithms can substantially reduce energy consumption. C# in some points showed an increased energy usage of 400% compared to the other programming languages as seen in tables 1-3 highlighting the importance of language choice in developing energy-efficient applications. These differences might not be so big in such a small scale but when the amount of data grows and the application becomes larger these differences can be detrimental to the sustainability of a product.

7.2 Project/Work method discussion

The work method chosen for this project was a design science inspired structure as can be seen in chapter 3.2. This method gave a clear and easy structure to follow. By identifying the problems in the first phase to understand the problem and what objectives that should be defined and what requirements needed in the second phase to create a relevant project. The design phase was a very important part of the method where all the requirements needed were designed and implemented. Then all the experiments were executed and later evaluated and written in the project report. This step by step method made the work coherent because each phase played an important role in the next phase. When each phase was finished the phase after became easier due to the information and knowledge gained.

The metrics used for the measurements in this case energy in Joule as the primary metric and the time as the secondary metric gave the perfect information and measurements to draw conclusions on all three scientific questions.

The chosen approach was something that could have been better. The work needed more algorithms to get more broad and accurate results. More tests with more datasets and more iteration to get better results and more accurate correlations. The jump between the amount of data was very big and it would be better to have smaller increases.

The methodological challenges were ensuring consistent testing conditions, as background processes and hardware variability could affect results. Measuring energy usage in a real-world setting could provide further validation of the test results.

7.3 Scientific Discussion

This work contributes to the field of sustainable computing by providing data on energy efficiency across different programming languages and algorithms. The work also shows that time and energy consumption are closely related where the relation is a 1 to 1 relation making it easier for developers to create green software by optimizing performance and reducing execution time of a program.

The scientific question can all be answered with the achieved result. The first question is that the energy consumption grows along the computational complexity of an algorithm which highlights that developers should use algorithms with better algorithmic complexity and try to optimize them for further gains. The answer to the second scientific question is a 1 to 1 relationship which can be further confirmed by the formula $E(J) = P(W) * t(s)$ indicating that the time and energy consumed grows the same amount given that the Wattage stays the same. The third scientific question got the most compelling results where it is very easy to see the energy efficiency of the different programming languages and that choosing the right language can significantly reduce energy consumption of an application or system. When looking at the answer of question three it is easy to see that Java and Go are very well optimized and energy efficient with very good consistency. C# which is a Java like language that does not perform as well and reduces energy efficiency by a significant amount even in executing the easier and less demanding tasks.

The knowledge gained by this work is not broad due to the tests being limited to only three algorithms and three programming languages. This does not generalize well due to the possibility of the programming languages to be more or less energy efficient during different tasks. Sorting algorithms are a very small part of software and do not accommodate the whole product. More algorithms with different time complexities are needed to see a better correlation and result to make it possible to draw better conclusions. This knowledge can after all be used to highlight the importance of choosing the right tool for the job and to think more of programming languages as a tool and

less a preference giving developers more knowledge across different languages and tools.

7.4 Consequence analysis/Recommendation

The contribution of this work is to give insights into how energy can be saved through choosing the right algorithms, programming language and optimization creating better performing software that can be sustainable and leave minimal impact on the environment. Increasing the lifecycle of both software and hardware.

I recommend that companies consider adopting more energy-efficient practices, tools and methods to optimize performance, energy efficiency and as a result get savings in cost. Organizations and developers should integrate energy efficiency and green testing into their software development lifecycle, from initial design to deployment and maintenance. Creating green and efficient products.

7.5 Ethical and societal discussion

From a human perspective reducing the energy consumption of software applications contributes to broader environmental goals benefiting society as a whole. Developers have a responsibility to consider the environmental impact of their work. Green Coding can lead to significant reductions in carbon footprints, combating climate change.

From a privacy perspective it is important that energy efficient practices do not compromise user privacy or data security.

By highlighting the importance of energy efficient coding, this project encourages a shift towards more sustainable technology practices. This shift can drive innovation in the field of green computing and inspire developers and organizations to adopt more environmentally friendly approaches on all fronts.

8. Conclusions

This project investigated the impact of green coding on software development to reduce environmental footprint. The main goals were to analyze algorithmic complexity, execution time and the energy consumption of different programming languages. The scientific questions posed were the following.

1. **How does algorithmic complexity affect energy consumption of an algorithm?**

The results of the work indicates that the energy consumption of an algorithm grows in the same manner as the algorithmic complexity. In conclusion, when the amount of data given to an algorithm with the time complexity of $O(n^2)$ doubles should the energy consumption quadruple. The algorithmic complexity gives a good understanding to how much time the algorithm needs to execute and how that time and energy grow.

2. **What is the correlation between execution time and energy usage?**

Time and energy usage are directly related with a 1:1 relation meaning that the energy consumption grows proportionally to the growth of the time. Meaning that performance optimization is also an energy consumption optimization.

3. **Do different programming languages consume more energy for the same algorithm?**

Different programming languages have different energy consumption metrics. Each language performs differently from the other depending on the layers of abstraction, compiler optimization and other factors. Meaning that programming languages should be picked carefully.

The project's scientific contribution lies in its empirical analysis of energy consumption across different algorithms and languages, providing a valuable resource for developers aiming to adopt greener coding practices. By demonstrating the environmental benefits of optimized code, this work encourages the software industry to prioritize sustainability.

8.1 Future work

For future work a real life project can be built, deployed and tested in a real world scenario. The energy consumption of the project can be noted to then try to optimize the application to reduce the energy consumption. Through enough optimization a result can be given to how much a software product can be optimized and what techniques and methods should be used. This can create a guideline for developers of techniques and methods to consider when writing green software.

References

- [1] Emmanuel Okewu *et al.*, “Optimizing green computing awareness for environmental sustainability and economic security as a stochastic optimization problem,” *Sustainability*. Vol 9(10), 2017.
<https://www.mdpi.com/2071-1050/9/10/1857>
- [2] Jason Grealley *et al.*, “Green Algorithms: Quantifying the Carbon Footprint of Computation,” *Advanced Science*., vol. 8, no. 12, May 2021 <https://onlinelibrary.wiley.com/doi/full/10.1002/advs.202100707>
- [3] Hannah Ritchie. “Sector by sector: where do global greenhouse gas emissions come from?” *Our World in Data*(2020). URL: https://ourworldindata.org/ghg_emissions-by-sector
- [4] Shalabh Agarwal, Asoke Nath and Dipayan Chowdhury “ Sustainable Approaches and Good Practices in Green Software Engineering,” *International Journal of Research and Reviews in Computer Science*., Vol. 3, no. 1 February 2012
- [5] A. Hoheisel, B. Betterman, I. Dunham, H. O’Leary and M. Mahmoud, "Green Computing from a Holistic Perspective," *2022 International Conference on Computational Science and Computational Intelligence (CSCI)*. URL: [Green Computing from a Holistic Perspective | IEEE Conference Publication | IEEE Xplore \(miun.se\)](#)
- [6] J. Light, "Energy usage profiling for green computing," *2017 International Conference on Computing, Communication and Automation (ICCCA)*. URL: [Energy usage profiling for green computing | IEEE Conference Publication |](#)
- [7] “Press corner,” *European Commission - European Commission*.
https://ec.europa.eu/commission/presscorner/detail/en/QANDA_22_6229
https://ec.europa.eu/commission/presscorner/detail/en/qanda_22_6229
- [8] Sara S.Mahmoud and Imtiaz Ahmad. “A Green Model For Sustainable Software Engineering” *2013 International Journal of Software Engineering and Its Applications*. URL: [document \(psu.edu\)](#)

- [9] J. Corral-Garcia, F. Lemus-Prieto, J.L. Gonzalez-sanches and M.A Perez-Toledano. "Analysis of Energy Consumption and Optimization Techniques for Writing Energy-Efficient Code" 2019 *Electronics*. URL: [Electronics | Free Full-Text | Analysis of Energy Consumption and Optimization Techniques for Writing Energy-Efficient Code \(mdpi.com\)](https://www.mdpi.com/electronics/8/12/1947)
- [10] K. Säfsten och M. Gustavsson, *Forskningsmetodik för ingenjörer och andra problemlösare*, 2 Uppl. Studentlitteratur AB. 2019
- [11] PowerJoular and JoularJX: Multi-Platform Software Power Monitoring Tools. Adel Nouredine. In the 18th International Conference on Intelligent Environments (IE2022). Biarritz, France, 2022.
- [12] "Quicksort Pseudocode :: CC 310 Textbook," [textbooks.cs.ksu.edu. https://textbooks.cs.ksu.edu/cc310/7-searching-and-sorting/19-quicksort-pseudocode/index.html](https://textbooks.cs.ksu.edu/cc310/7-searching-and-sorting/19-quicksort-pseudocode/index.html)
- [13] "Merge Sort Pseudocode :: CC 310 Textbook," [textbooks.cs.ksu.edu.](https://textbooks.cs.ksu.edu/cc310/7-searching-and-sorting/10-selection-sort-pseudocode/index.html)
- [14] "Selection Sort Pseudocode :: CC 310 Textbook," [textbooks.cs.ksu.edu. https://textbooks.cs.ksu.edu/cc310/7-searching-and-sorting/10-selection-sort-pseudocode/index.html](https://textbooks.cs.ksu.edu/cc310/7-searching-and-sorting/10-selection-sort-pseudocode/index.html)
- [15] BillWagner, "Overview - A tour of C#," [learn.microsoft.com](https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview), May 08, 2024.<https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview>[Overview - A tour of C# | Microsoft Learn](https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview)
- [16] "go.dev," [go.dev. https://go.dev/](https://go.dev/) [The Go Programming Language](https://go.dev/)
- [17] "Java Programming Language," [docs.oracle.com. https://docs.oracle.com/javase/8/docs/](https://docs.oracle.com/javase/8/docs/)
- [18] Statista, "Most Used Languages among Software Developers Globally 2019," *Statista*, 2023 <https://www.statista.com/statistics/793628/worldwide-developer-survey-most-used-languages/>

Appendix A: Source Code

[TahaKhudher/exjobbet \(github.com\)](https://github.com/TahaKhudher/exjobbet)